GIT & GITHUB

Interactive Developer Notes • Commands • Branching • Stashing • Tags • Rebase

Quick navigation • Configuration • Core commands • Branching • Stashing • Best practices • Tags • Rebase

1. Git Configuration

Identity Configuration

Set the name and email Git will attach to your commits.

```
git config --global user.email "ishwarsing.codes@gmail.com"

git config --global user.name "ishwarsingh"
```

Tip: Check your current global configuration with **git config --global --list**.

2. Core Git Commands

Repository & Working Tree

Command	Purpose
git init	Initialize a new Git repository.
git status	See staged, unstaged, and untracked changes.
git add / git add .	Move changes into the staging area.
git log	View commit history.
git log --oneline	View a compact one-line commit history.
git restore --staged	Remove a file from staging without deleting its changes.
.gitkeep	Convention used to make an otherwise-empty directory trackable.

Important: Staging is not committing. Closing Git Bash does not remove staged changes; they remain in your local repository.

3. Branching in Git

Every feature gets its own line of development

Command	What it does
git branch	Create a branch.
git switch	Switch to an existing branch.
git switch -c	Create a new branch and switch to it immediately.
git merge	Merge the specified branch into the branch you are currently on.
git branch -d	Delete a local branch after it has been merged.

Typical flow

```
git switch -c feature/user-login

git add .

git commit -m "Add user login"

git switch main

git merge feature/user-login
```

When you merge a branch, Git combines its commits into the current branch. The exact history depends on whether the merge can be fast-forwarded or creates a merge commit.

4. Stashing in Git

Temporarily put unfinished work aside

Command	Purpose
git stash	Temporarily save working changes and clean the working tree.
git stash list	List saved stashes.
git stash pop	Apply the most recent stash and remove it from the stash list if successful.
git stash apply	Apply a stash while keeping that stash in the stash list.

Memory trick: **pop** = apply + remove; **apply** = apply + keep.

5. Branching Best Practices

A clean branching strategy

1. One Feature = One Branch

Create a dedicated branch for each feature, bug fix, or improvement.

```
feature/user-login
```

2. Never Develop Directly on main

Protect main and keep it stable. Use Pull Requests for changes.

```
main → feature branch → PR → main
```

3. Delete Merged Branches

Once a branch is merged, delete it to keep the repository clean.

```
git branch -d feature/user-login
```

4. Use Meaningful Branch Names

Use clear, descriptive names that indicate the purpose.

```
feature/payment-integration • bugfix/login-error • hotfix/security-issue
```

5. Keep Branches Short-Lived

Merge changes as soon as practical to reduce divergence and merge conflicts.

```
Create → work → PR → merge → delete
```

Result: Good branching habits lead to a cleaner history, fewer conflicts, and a healthier codebase.

6. Git Tags

Mark important commits — especially releases

A tag is a name that points to a specific commit. Unlike a normal branch, a release tag is intended to stay attached to that commit.

Two common types

Type	Command	Use
Annotated	git tag -a v1.0 -m "my-release-1"	Recommended for releases; stores tag metadata/message.
Lightweight	git tag v1.0	Simple pointer to a commit.

Useful commands

```
git tag -a v1.0 -m "my-release-1"

git tag

git tag v1.0 # lightweight tag

git push origin v1.0

git push origin --tags
```

Note: A normal **git push origin main** does not automatically push your local tags. Push a specific tag or use **git push origin --tags**.

7. Git Rebase

Replay your branch commits on a new base

Rebase takes the commits unique to your branch, temporarily removes them, moves your branch to the new base, and then replays those commits on top.

```
# On your feature branch

git rebase main
```

Before:

```
A — B — C main

\

D — E feature
```

After:

```
A — B — C — D' — E'

↑

feature
```

D' and E' are new commit objects because their parent/base changed.

Rebase vs Merge

```
MERGE: A — B — C ——— M

\ /

D — E ——


REBASE: A — B — C — D' — E'
```

Conflict workflow

```
git rebase main

git status

git add <resolved-file>

git rebase --continue

# If you want to cancel:

git rebase --abort
```

Golden rule: Avoid rebasing shared/public branches that other developers are already building on. Rebase rewrites commit history. If you must update a remote branch after rebasing, prefer **git push --force-with-lease** over **git push --force**.

8. One-Page Git Workflow

The mental model

```
WORKING DIRECTORY

↓

git add

↓

STAGING AREA

↓

git commit

↓

LOCAL REPOSITORY

↓

git push

↓

REMOTE REPOSITORY (GitHub)
```

Feature workflow

```
git switch main

git pull origin main

git switch -c feature/my-feature

# make changes

git status

git add .

git commit -m "Implement my feature"

git push -u origin feature/my-feature

# Open a Pull Request on GitHub
```

Quick reminders

• Staged ≠ committed — staged changes live in the index until committed.
• Push ≠ commit — push uploads existing commits to a remote.
• Branch ≠ tag — branches move; release tags normally stay at a specific commit.
• Merge ≠ rebase — merge joins histories; rebase rewrites/replays your branch commits on a new base.